

Big Data: Management and Analytics

For this task you will use SparkContext in PySpark. For more information about PySpark read the documentation here <https://spark.apache.org/docs/latest/api/python/index.html> Make sure to have Java ≥ 1.8 installed. You can download the text file "Text-Data.txt", which is needed for task c and d.

(a) Start by creating a list of 0 to 20000, afterwards use `parallelize()` to make an RDD from the list (distribute your list into 4 partitions). Use RDD's filter and a lambda function to convert your list into a list of odd numbers. Show the result by using the `take()` function and output the first five numbers in the odds list.

```
import findspark
findspark.init()
from pyspark import SparkContext
sc = SparkContext.getOrCreate()
collect_rdd = sc.parallelize(range(20000), 4)

print(collect_rdd.filter(lambda x: x%2 != 0).take(5))
collect_rdd.getNumPartitions()
```

```
In [5]: import findspark
        findspark.init()
```

```
In [4]: from pyspark import SparkContext
        sc = SparkContext.getOrCreate()
```

```
In [29]: collect_rdd = sc.parallelize(range(20000), 4)

        print(collect_rdd.filter(lambda x: x%2 != 0).take(5))
        collect_rdd.getNumPartitions()
```

```
[1, 3, 5, 7, 9]
```

```
Out[29]: 4
```

(b) Explain what the parallelize-function does, explain RDD, and the procedure in your code. Explain why it is different from what we did in the first assignment when we asked to filter even numbers in a list. (What is the difference between python's filter-function and the RDD-filter-function?)

RDD stands for, Resilient Distributed Datasets (RDD) this is the fundamental data structure in pyspark. Each record in the RDD is divided into logical partitions that can be calculated on the various nodes of the cluster. RDD is an immutable distributed collection of objects

We can create an RDD using parallelize function in pyspark, from an existing list. PySpark shell provides SparkContext variable "sc", use sc.parallelize() to create an RDD.

With this statement we can create a new RDD.

In the above program in question (a), we have imported sparkContext from pyspark and the sparkContext is noted with a variable "sc".

collect_rdd = sc.parallelize(range(20000), 4), here we are creating a collect_rdd a RDD using parallelize function which have range from 0 to 20000, and partitions of 4.

Partitions are basic units of parallelism in PySpark. RDDs in PySpark are a collection of partitions.

In the next step is print(collect_rdd.filter(lambda x: x%2 != 0).take(5)), here the filter function and lambda is used, for finding out the odd numbers in the collect_rdd. And take(5) is used to just print 5 elements.

collect_rdd.getNumPartitions() this statement prints the number of partitions in the RDD.

The filter() method filters the given sequence with the help of a function that tests each element in the sequence to be true or not.

In the first assignment filter function returned true when the $x \bmod 2 == 0$ thus the even list is printed. PySpark filter() function is used to filter the rows from RDD/DataFrame based on the given condition here the given condition using lambda is to filter out the odd numbers and take (5) elements

(c) Count the number of appearance of each word in the txt-file "Text-Data" using PySpark. Show the results of the 25 words with the most occurrences as a tuple (e.g. ("word", 23)).

```
from pyspark import SparkContext
sc = SparkContext.getOrCreate()

# Read the input file and Calculating words count
text_file = sc.textFile("Text-data.txt")
counts = text_file.flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda x, y: x + y)
output = counts.collect()
bSorted = counts.sortBy(lambda a: -a[1])
aSorted = bSorted.take(25)
print("The 25 words with most occurrence is : ", aSorted)
sc.stop()
```

```
In [24]: from pyspark import SparkContext
sc = SparkContext.getOrCreate()

# Read the input file and Calculating words count
text_file = sc.textFile("Text-data.txt")
counts = text_file.flatMap(lambda line: line.split(" ")) \
                  .map(lambda word: (word, 1)) \
                  .reduceByKey(lambda x, y: x + y)

output=counts.collect()
bSorted = counts.sortBy(lambda a: -a[1])
aSorted=bSorted.take(25)
print("The 25 words with most occurence is : ",aSorted)

sc.stop()
```

The 25 words with most occurence is : [(' ', 367757), ('is', 144241), ('a', 76862), ('the', 75044), ('of', 70819), ('to', 42996), ('or', 35374), ('in', 31339), ('and', 26971), ('an', 14529), ('that', 13799), ('with', 13296), ('by', 11308), ('for', 10999), ('on', 7388), ('from', 6766), ('as', 6671), ('who', 5969), ('having', 5803), ('used', 4783), ('was', 4346), ('he', 3947), ('his', 3841), ('at', 3600), ('not', 3552)]

(d) List all those words, which can be classified as "auxiliary verbs" or "function words" to exclude them from your results (e.g. "by", "to", "and" are "not-meaningful words"). Order each "meaningful" word by occurrence.

```
from pyspark import SparkContext
sc = SparkContext.getOrCreate()
New_RDD = sc.textFile("Text-data.txt")
def similar(lines):
    lines = lines.lower()
    lines = lines.split()
    return lines
Split_rdd = New_RDD.map(similar)
stopwords=['a','all','the','as','is','am','an','and','be','been','from','had','I','I'd','why','with','was','wh',
'o','of','when','the','to','each','by','where','there','in','are']
RDD_words = New_RDD.flatMap(similar)
RDD_Final = RDD_words.filter(lambda x: x not in stopwords)
RDD_Final.sortByKey(False)
RDD_Final.take(25)
```

```

In [19]:
from pyspark import SparkContext
sc = SparkContext.getOrCreate()

New_RDD = sc.textFile("Text-data.txt")
def similar(lines):
    lines = lines.lower()
    lines = lines.split()
    return lines
Split_rdd = New_RDD.map(similar)

stopwords = ['a', 'all', 'the', 'as', 'is', 'am', 'an', 'and', 'be', 'been', 'from', 'had', 'I', 'I'd', 'why', 'with', 'was', 'who', 'of', 'when', 't
RDD_words = New_RDD.flatMap(similar)
RDD_Final = RDD_words.filter(lambda x: x not in stopwords)

RDD_Final.sortByKey(False)
RDD_Final.take(25)

```

```

Out[19]: ['first',
'secretary',
'agriculture',
'norman',
'j.',
'colman',
'appointed',
'cleveland',
'first',
'secretary',
'energy',
'james',

```

Task 2

you will practice data management with Apache Spark. For more information about RDDs, DataFrames and Datasets in Apache Spark, read the documentation here <http://spark.apache.org/docs/latest/sql-programming-guide.html>. You can download the csv-file "DiscounterStock.csv" .

(a) What are the major differences between Resilient Distributed Dataset (RDDs), Data Frames and Datasets in Apache Spark?

RDDs	Dataframes	Datasets
RDD is an unstructured distributed collection of data elements.	It is the dispersed collection, which is divided into named columns.	It's a Dataframes extension with additional features like as type safety and an object-oriented interface.
RDDs have no built-in optimization engine. Developers must write their own optimized code.	It optimizes using a catalyst optimizer.	For optimization, it additionally employs a catalyst optimizer.

The schema must be explicitly defined here.	It will automatically determine the dataset's schema.	Using the SQL Engine, it will automatically discover the dataset's schema.
When it comes to simple tasks like grouping data, RDD is slower than both Dataframes and Datasets.	It has a simple API for aggregate. Both RDDs and Datasets are outperformed by it in terms of aggregation.	RDDs are faster than Dataframes, although Datasets are slightly slower.

(b) Load the csv-file "DiscounterStock.csv" in Python as a dataframe. What are the column-names? Print the first five column-names.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)
df.columns
df2 = df.select(df.columns[:5])
df2.show(10)
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import mean, min, max
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()

df = spark.read.csv('DiscounterStock.csv',header=True,inferSchema=True)
df.columns[:5]
```

```
['Date', 'Open', 'High', 'Low', 'Close']
```

```
In [77]: df2 = df.select(df.columns[:5])
df2.show(10)
```

Date	Open	High	Low	Close
2012-01-03	59.970001	61.060001	59.869999	60.330002
2012-01-04	60.209998999999996	60.349998	59.470001	59.709998999999996
2012-01-05	59.349998	59.619999	58.369999	59.419998
2012-01-06	59.419998	59.450001	58.869999	59.0
2012-01-09	59.029999	59.549999	58.919998	59.18
2012-01-10	59.43	59.709998999999996	58.98	59.040001000000004
2012-01-11	59.060001	59.529999	59.040001000000004	59.400002
2012-01-12	59.790001000000004	60.0	59.400002	59.5
2012-01-13	59.18	59.610001000000004	59.009997999999996	59.540001000000004
2012-01-17	59.869999	60.110001000000004	59.52	59.849998

only showing top 10 rows

(c) How can you display the schema of the dataframe of task (a)? Print out the statistics for each column.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)
```

```
df.printSchema()
df.describe().show()
```

```
In [103]: from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)
df.printSchema()
df.describe().show()
```

```
root
 |-- Date: string (nullable = true)
 |-- Open: string (nullable = true)
 |-- High: string (nullable = true)
 |-- Low: string (nullable = true)
 |-- Close: string (nullable = true)
 |-- Volume: string (nullable = true)
 |-- Adj Close: string (nullable = true)
```

	summary	Date	Open	High	Low	Close	Volume	Adj Close
count	1258		1258	1258	1258	1258	1258	1258
mean	null	72.35785375357709	72.83938807631165	71.9186009594594	72.38844998012726	8222093.481717011	67.2388384872814	
stddev	null	6.76809024470826	6.768186808159218	6.744075756255496	6.756859163732991	4519780.8431556	6.72260944999685	
min	2012-01-03	56.389998999999996	57.060001	56.299999	56.419998	10010500	50.36368	
max	2016-12-30	90.800003	90.970001	89.25	90.470001	9994400	84.9142160000000	

(d) Create a new data frame, which is a copy of the one in (a). Add a column called HV Ratio, which is the ratio of the High Price versus volume of stock traded for a day.

Code

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("how to read csv file").getOrCreate()

df = spark.read.csv('DiscounterStock.csv',header=True)

df2 = df.withColumn("HV Ratio",df["High"]/df["Volume"])

df2.select('Date','High','Volume','HV Ratio').show()
```



```
In [106]: from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)

df2 = df.withColumn("HV Ratio",df["High"]/df["Volume"])

df2.select('Date','High','Volume','HV Ratio').show()
```

Date	High	Volume	HV Ratio
2012-01-03	61.060001	12668800	4.819714653321546E-6
2012-01-04	60.349998	9593300	6.290848613094555E-6
2012-01-05	59.619999	12768200	4.669412994783916E-6
2012-01-06	59.450001	8069400	7.367338463826307E-6
2012-01-09	59.549999	6679300	8.915604778943901E-6
2012-01-10	59.709998999999996	6907300	8.644477436914568E-6
2012-01-11	59.529999	6365600	9.351828421515645E-6
2012-01-12	60.0	7236400	8.29141562102703E-6
2012-01-13	59.610001000000004	7729300	7.712212102001476E-6
2012-01-17	60.110001000000004	8500000	7.071764823529412E-6
2012-01-18	60.029999	5911400	1.015495466386981E-5
2012-01-19	60.73	9234600	6.576354146362592...
2012-01-20	61.25	10378800	5.90145296180676E-6
2012-01-23	60.98	7134100	8.547679455011844E-6
2012-01-24	62.0	7362800	8.420709512685392E-6
2012-01-25	61.610001000000004	5915800	1.041448341728929...
2012-01-26	61.84	7436200	8.316075414862431E-6
2012-01-27	61.119999	6287300	9.721183814992126E-6
2012-01-30	61.32	7636900	8.029436027707578E-6
2012-01-31	61.57	9761500	6.307432259386365E-6

only showing top 20 rows

(e) What is the max and min of the Volume column?

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import mean, min, max
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)
result = df.select([min("Volume"), max("Volume")])
result.show()
```

```
In [92]: from pyspark.sql import SparkSession
from pyspark.sql.functions import mean, min, max
spark = SparkSession.builder.appName("how to read csv file").getOrCreate()
df = spark.read.csv('DiscounterStock.csv',header=True)

result = df.select([min("Volume"), max("Volume")])
result.show()
```

min(Volume)	max(Volume)
10010500	9994400